

Esercitazione 5

📅 Glorno	March 26, 2025
☰ Riassunto	Esercitazione gRPC

1. Introduzione alla Comunicazione Distribuita

- **Concetto di sistema distribuito:**

Si parte dalla definizione di sistema distribuito, in cui vari processi (che possono risiedere su macchine diverse) devono comunicare, sincronizzarsi e cooperare.

- **Livelli di comunicazione:**

Il documento sottolinea come il livello più basso di comunicazione in un sistema distribuito sia lo scambio di messaggi. A questo livello vengono implementate interfacce che semplificano l'uso, come:

- Remote Procedure Call (RPC)
- Active messages
- Publish/subscribe
- Streams

- **Tipi di comunicazione:**

Vengono poi distinti due modi fondamentali di comunicare:

- **Sincrono:** il mittente attende che il messaggio arrivi a destinazione, bloccandosi fino al completamento.
- **Asincrono:** il mittente continua la propria esecuzione senza attendere la conferma della ricezione.

Questi meccanismi possono essere ulteriormente combinati con comunicazioni **persistenti** (il messaggio viene memorizzato fino a quando il destinatario non lo riceve) o **transitorie** (la consegna avviene solo se mittente e destinatario sono attivi contemporaneamente).

2. Interfacce RPC e il Caso MPI

- **Interfacce per RPC:**

Si introduce MPI (Message Passing Interface), uno standard per la programmazione parallela e distribuita, basato sul passaggio di messaggi.

- **Struttura di un programma MPI:**

Un'applicazione MPI è composta da processi concorrenti che eseguono un programma sequenziale ma comunicano attraverso chiamate specifiche della libreria MPI.

Operation	Description
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until transmission starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

- **Esempi di funzioni MPI:**

- **Punto a punto:**

`MPI_Send` e `MPI_Recv` per inviare e ricevere messaggi.

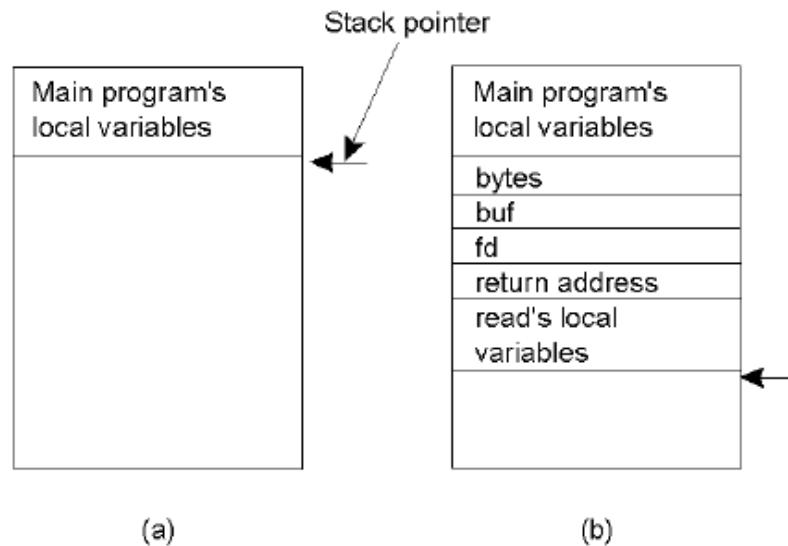
- **Comunicazione collettiva:**

Funzioni come `MPI_Bcast` per la diffusione dei messaggi e `MPI_Gather` per raccogliere dati da più processi.

3. Introduzione al Remote Procedure Call (RPC)

- **Concetto di RPC:**

Il RPC è una tecnica che permette a un client di invocare una procedura (o metodo) su un server remoto, quasi come se si trattasse di una normale chiamata locale.



- **Differenze rispetto alle chiamate locali:**

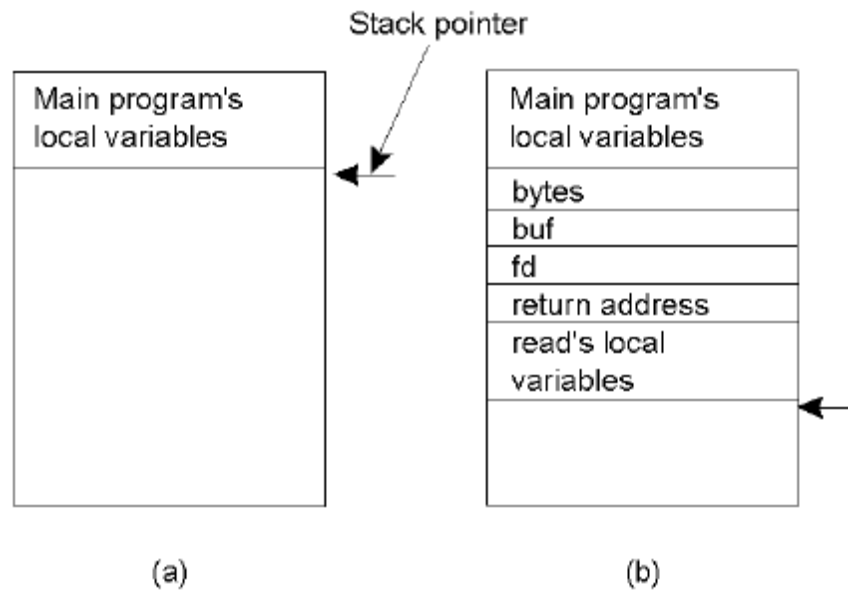
La differenza principale sta nel fatto che il codice della procedura viene eseguito su un'altra macchina o processo, il che richiede un meccanismo di marshalling (ossia la conversione e il trasporto dei parametri) e di demarshalling dei dati.

- **Esempi pratici:**

Vengono menzionati vari esempi, come Unix RPC, Java RMI e gRPC.

4. Meccanismo di funzionamento di una chiamata RPC

Il documento descrive in dettaglio il flusso di una chiamata RPC, evidenziando i seguenti passaggi:



- **Stub Client:**

Quando il client invoca una chiamata, il client stub (una funzione che gestisce il marshalling) prepara i parametri e li incapsula in un messaggio.

- **Trasmissione del messaggio:**

Il messaggio viene passato al sistema operativo locale, che lo invia al sistema operativo remoto attraverso la rete.

- **Stub Server:**

Una volta che il messaggio raggiunge il server, il server stub estrae i parametri e invoca la procedura richiesta.

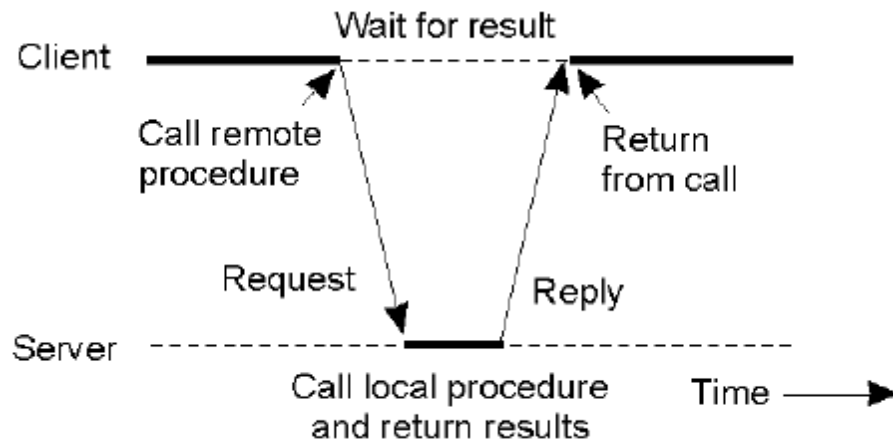
- **Esecuzione e risposta:**

Dopo aver eseguito la procedura, il server restituisce il risultato che viene incapsulato in un nuovo messaggio e inviato indietro al client, dove il client stub lo estrae e lo restituisce all'applicazione.

5. RPC Sincroni e Asincroni

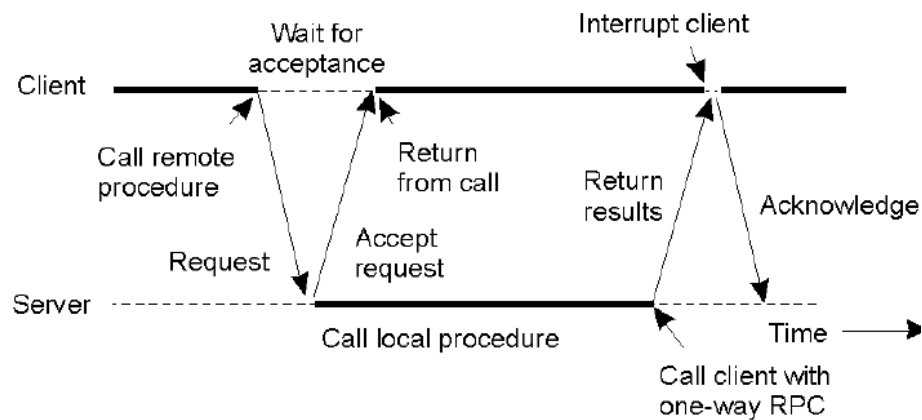
- **RPC Sincrono:**

Il client attende la risposta del server prima di continuare, rendendo l'operazione simile a una tradizionale chiamata di funzione.



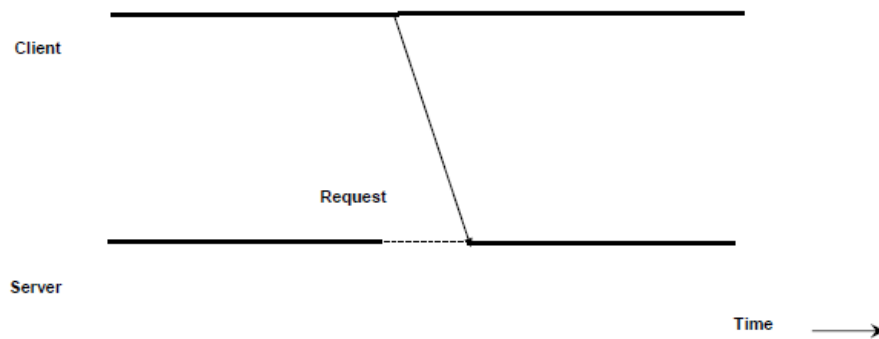
- **RPC Asincrono:**

Il client invia la richiesta e continua l'esecuzione senza attendere immediatamente la risposta.



- **RPC One-Way (asincrono):**

Una variante in cui il client invia la richiesta senza aspettare conferma, il che implica che non è garantita la ricezione della risposta.



6. Introduzione a gRPC

- **Cosa è gRPC:**

gRPC è presentato come un framework moderno, open source e ad alte prestazioni per realizzare RPC in ambienti distribuiti.

- **Vantaggi di gRPC:**

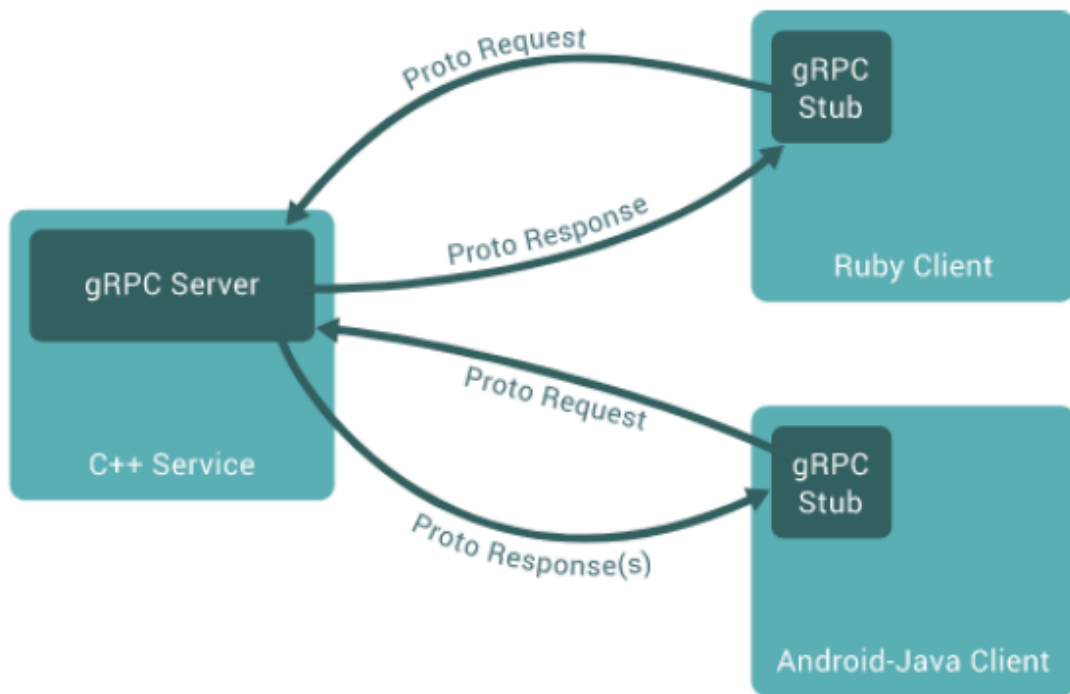
- Alta efficienza e prestazioni.
- Supporto per ambienti eterogenei: client e server possono essere scritti in linguaggi diversi.
- Integrazione con strumenti per il load balancing, tracing, health checking e autenticazione.

- **Utilizzo di Protocol Buffers:**

gRPC sfrutta i Protocol Buffers sia come linguaggio di definizione dell'interfaccia (IDL) sia come formato per lo scambio di messaggi, garantendo così una serializzazione efficiente.

7. Panoramica di gRPC e dei suoi Componenti

- **Modello Client-Server in gRPC:**



- **Sul lato server:**

Il server implementa le interfacce definite nel file `.proto` e avvia un server gRPC per gestire le richieste.
- **Sul lato client:**

Il client usa uno stub che espone le stesse interfacce del server, permettendo di effettuare chiamate come se fossero locali.
- **Ambienti e linguaggi:**

gRPC è progettato per funzionare in ambienti molto vari, da sistemi interni a Google fino a desktop e dispositivi mobili, supportando numerosi linguaggi di programmazione.

8. Introduzione ai Protocol Buffers

- **Definizione di un file `.proto`:**

Il primo passo per utilizzare Protocol Buffers è definire in un file di testo (`.proto`) la struttura dei dati da serializzare.

- **Struttura dei messaggi:**

I dati sono organizzati in “messaggi”, che contengono coppie nome-valore.

- Esempio: un messaggio `Person` con campi come `name`, `id` e un flag booleano.

- **Compilazione del file .proto:**

Utilizzando il compilatore `protoc`, vengono generati i file di codice (classi e metodi) per il linguaggio scelto. Questi file consentono di accedere ai dati (setter/getter) e di serializzarli o deserializzarli.

9. Definizione del Servizio in gRPC

- **Descrizione del file .proto per un servizio:**

Oltre alla definizione dei messaggi, il file .proto include la definizione del servizio gRPC, specificando i metodi remoti e i relativi parametri e tipi di ritorno.

- **Esempio pratico:**

Viene mostrato un esempio di definizione di un servizio `Greeter` con un metodo `SayHello` che prende in input un `HelloRequest` e restituisce un `HelloReply`.

- **Generazione del codice:**

Utilizzando il plugin gRPC per il compilatore, il file .proto genera sia il codice per il client che per il server, oltre a quello standard dei Protocol Buffers.

10. Tipologie di RPC in gRPC

gRPC supporta quattro tipologie di metodi RPC:

- **Unary RPC:**

Il client invia una singola richiesta e riceve una singola risposta, simile a una normale chiamata di funzione.

- **Server Streaming RPC:**

Il client invia una richiesta e il server risponde con una sequenza di messaggi, che il client legge fino alla fine dello stream.

- **Client Streaming RPC:**

Il client invia una serie di messaggi e, una volta completata l'invio, attende una risposta singola dal server.

- **Bidirectional Streaming RPC:**

Sia il client che il server inviano messaggi in maniera indipendente attraverso uno stream bidirezionale, mantenendo l'ordine dei messaggi in ciascuno stream.

11. Uso dell'API gRPC e il Ciclo di Vita di una Chiamata

- **Implementazione lato server e client:**

- **Server:** implementa i metodi definiti, gestisce il parsing delle richieste e l'invio delle risposte.
- **Client:** usa lo stub generato per invocare le chiamate remote, gestendo la serializzazione e deserializzazione dei messaggi.

- **Differenza tra modalità sincrona e asincrona:**

Le API gRPC offrono entrambe le modalità, permettendo al client di bloccare in attesa della risposta o di proseguire l'esecuzione in maniera asincrona.

12. Dettagli sul Ciclo di Vita di una Chiamata RPC

- **Fasi di una Unary RPC:**

1. Il client invoca lo stub, passando metadata e parametri.
2. Il server riceve i dati e, eventualmente, invia metadati iniziali.
3. Il server elabora la richiesta e invia il risultato insieme a informazioni di stato e metadati finali.
4. Il client riceve la risposta e conclude la chiamata.

- **Server Streaming, Client Streaming e Bidirectional Streaming:**

Vengono spiegate le differenze nel flusso di messaggi e nella gestione degli stream, evidenziando come gRPC garantisca l'ordine dei messaggi all'interno di una singola chiamata.

13. Gestione di Deadlines, Timeout e Cancellazioni

- **Deadlines/Timeouts:**

gRPC permette al client di specificare un tempo massimo di attesa (deadline) per la risposta del server. Se la risposta non arriva in tempo, la chiamata viene terminata con un errore (`DEADLINE_EXCEEDED`).

- **Cancellazione delle RPC:**

Sia il client che il server possono decidere di cancellare una chiamata in corso se, ad esempio, non è più necessaria o se si verificano errori.

- **Utilizzo dei metadata:**

Informazioni aggiuntive (come dettagli di autenticazione o altri parametri) possono essere associate alle chiamate sotto forma di coppie chiave/valore.

14. Gestione dei Canali di Comunicazione

- **Definizione di un channel in gRPC:**

Un channel rappresenta una connessione a un server gRPC, definita da host e porta.

- **Configurazione e gestione:**

I client possono impostare vari parametri (come la compressione dei messaggi) e monitorare lo stato del channel (ad es. se è attivo o in idle).

- **Comportamento specifico per linguaggio:**

La gestione del ciclo di vita del channel può variare a seconda del linguaggio di programmazione utilizzato.